

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

Temporal Graph Convolutional Network for Implicit Relation Prediction: Leveraging Timestamps and Confidence

Lida Mirzaei^a, Ziad Kobti^a

^a*School of Computer Science - University of Windsor, 401 Sunset Ave, Windsor and ON N9B 3P4, Canada*

Abstract

In the dynamic landscape of social network analysis, the accurate prediction of implicit relationships presents a pivotal challenge. This paper introduces an innovative solution, the Relation Temporal Graph Convolutional Network with Confidence (R-CTGCN), specifically designed to address the intricate task of predicting implicit relations within evolving social networks. R-CTGCN unifies timestamp temporal embeddings, confidence metrics, and PFs within a comprehensive graph neural network framework, aiming to capture the evolving dynamics of networks and enhance predictive accuracy. Experimental evaluations conducted on diverse datasets, including Epinions and Enron, showcase R-CTGCN's superior performance compared to both baseline models and contemporary state-of-the-art methods. The emphasis on the roles of confidence and PFs underscores their significance in implicit relationship prediction. The outcomes contribute substantively to the understanding of predicting implicit relationships, positioning R-CTGCN as a robust tool tailored for complex social network scenarios.

© 2024 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems

Keywords: Implicit Relationship Prediction; Temporal Graph CN; Timestamp Embedding; heterogeneous network; Confidence; and PF.

1. Introduction

In the evolving landscape of social network analysis, the task of accurately predicting implicit relationships has emerged as a critical challenge [1]. Unlike explicit connections, which are directly observable, implicit relationships are inferred from patterns of interactions and behaviors within the network. These relationships, such as trust [3], influence [14, 15], or hierarchy [1], are pivotal in understanding the complex dynamics of social networks. Understanding and analyzing these implicit relationships pose significant challenges due to the intricate nature of social networks and the diverse characteristics of their constituent nodes [1]. The inherent limitations in data, marked by

* Corresponding author. Tel.: +1-647-891-6489 .

E-mail address: mirzaei4@uwindsor.ca

ambiguity, incompleteness, and metaphorical black-and-white quality, further complicate the task [1, 2]. While traditional studies have predominantly focused on explicit relationships, observable and quantifiable within a network, recent studies [1, 3, 5, 6] have revealed the importance of implicit relationships in networks, and developed models to predict and identify implicit relations. Some of these studies [1, 10] show that Graph Neural Networks (GNNs) have gained prominence in social network analysis for their ability to capture complex relationships within graph-structured data. Notable advancements include the Graph Convolutional Network (GCN), which has demonstrated success in tasks such as node classification and link prediction [10, 11, 12]. While these models excel in capturing local graph structures, the temporal dynamics inherent in social networks pose a challenge that traditional GNNs often struggle to address. Also, these methods often lack integration with confidence metrics, hindering their ability to capture the certainty or reliability of predicted relationships. For example, Mirror[1] approach uses metrics such as Jaccard similarity [17] and Adamic-Adar similarity [16] to provide insights into the topological proximity and shared neighbors within a graph. While these metrics contribute to local graph connectivity, their integration into a comprehensive framework for implicit relationship prediction remains an open challenge.

A fundamental issue in existing methods is the absence of common features or interactions across different networks and ignoring non-linear dependencies among features within social networks, impeding the applicability of attribute-based methods. Motivated by the gaps identified in existing literature, we introduce a novel approach, the Relation Temporal Graph Convolutional Network with Confidence (R-CTGCN), that seeks to integrate timestamp temporal embeddings, confidence metrics, and PFS within a unified GNN framework. This comprehensive approach aims to provide a nuanced understanding of implicit relationships in dynamic social networks, offering improved predictive accuracy and reliability. In summary, while existing works contribute valuable insights, the proposed R-CTGCN approach builds upon and extends these foundations to address the multifaceted challenges associated with implicit relationship prediction in evolving social networks.

Our proposed approach, R-CTGCN, makes several significant contributions to the field of implicit relationship prediction within dynamic social networks. R-CTGCN integrates temporal dynamics seamlessly into the model architecture. By incorporating timestamp-based temporal embeddings, the model gains a profound understanding of the evolving patterns within relationships over time. This temporal awareness enhances the accuracy of relationship predictions, especially in scenarios where the dynamics of connections play a crucial role. We introduce a confidence metric that combines Jaccard similarity (JS) and Adamic-Adar similarity (AS) measures and focuses on local graph connectivity, emphasizing shared neighbors in the relationship prediction process. The integration of confidence metrics into the model enhances its ability to discern the reliability and certainty of predicted relationships, a crucial aspect often overlooked in existing methodologies. To capture the intricate, non-linear dependencies among features, R-CTGCN introduces PF generation. By meticulously extracting PFs from hand-crafted features representing user interactions and timestamps, the model extends its capacity to understand and predict complex interactions within evolving networks. This contribution enables R-CTGCN to discern and represent non-linear relationships more comprehensively than traditional models. The architecture of R-CTGCN incorporates a specialized TGCNConv designed to consider both temporal features and confidence values of users. This advanced convolutional layer extends the Chebyshev Graph Convolutional Layer, providing a comprehensive view of evolving relationships over time. The inclusion of confidence values contributes to the improvement of prediction reliability, making the model more robust in real-world scenarios.

2. Related Works

The exploration of relationship identification across domains has seen pivotal contributions, focusing on enhancing methodological precision despite facing challenges [2, 7, 9]. Wren et al. [4] automated the discovery of biomedical relationships through co-occurrence networks, revealing novel connections but struggling with defining uninteresting relationships and text analysis reliance. Song et al. [9] improved gene relationship identification through the Gene-Citation-Gene network, noting limitations in data scope and urging further bioinformatics research. Peng et al. [7] combined Steiner Tree algorithms and clustering for biological network topology identification, facing scalability and robustness issues. Cen et al. [6] tackled trust prediction in E-Commerce with eTrust, confronting computational efficiency and scalability challenges. Liu et al. [2] explored academic network relationships with Shifu2, suggesting further investigation into temporal changes and ethical concerns. Wang et al. [5] aimed at scholar similarity for col-

laboration recommendations with ACNE, limited by data diversity and scalability. Barzegar Nozari and Koochi [3, 13] and Liu et al. [13] contributed to recommender systems and heterogeneous network relationship inference with ITN and MIRROR[1] models, respectively, pinpointing the need for scalability enhancements and overfitting avoidance. These endeavors highlight the continuous effort to refine relationship identification methods within varied research landscapes. Despite its effectiveness, considerations of scalability and strategies to avoid overfitting in limited data scenarios were highlighted as potential challenges. The comprehensive examination of these research endeavors reveals commendable strengths accompanied by inherent limitations, underscoring the ongoing challenge of refining the precision and applicability of relationship identification methodologies.

3. Proposed Approach

We present a novel approach for implicit relation prediction, leveraging a combination of a new confidence metric and timestamps into a GNN. The aim is to capture temporal dynamics in relational data, enhancing the predictive accuracy of relation outcomes. The approach is a comprehensive framework that integrates PF generation, temporal dynamics (timestamps), confidence, feature encoding, and graph-based learning. Each component and its corresponding mathematical formulation are designed to address specific challenges in dynamic relationship prediction. By combining these elements, the model offers a nuanced and sophisticated approach to understanding and forecasting complex interactions within evolving networks. The block diagram of the proposed method is illustrated in figure 1.

Confidence Metric: Prior research, as exemplified in studies[3, 13], and[17], has identified confidence as a pivotal element in forecasting trust dynamics within relational networks. Inspired by these findings, our method incorporates a confidence metric that amalgamates insights derived from two established similarity measures: Jaccard similarity (JS) as referenced in [17], and Adamic-Adar similarity (AS) as discussed in [16]. This integration is designed to augment the predictive efficacy of our model. Both JS and AS are instrumental in quantifying the degree of topological proximity and the extent of shared connections among nodes within a network, thereby offering a comprehensive view of relational structures. The combined confidence score C between users i and j is calculated as follows:

$$C_{ij} = \frac{2 \times JS_{ij} \times AS_{ij}}{JS_{ij} + AS_{ij}} \quad (1)$$

This formulation emphasizes the significance of common neighbors in determining the confidence score. By integrating JS and AS, confidence metric could capture both the broad structural context and the nuanced significance of specific types of connections in the network. This dual approach might be particularly effective in complex networks where relationships are not solely based on the quantity of shared connections but also on their nature and quality.

Polynomial Features: Here we explore the non-linear relationships by extracting PFs since there is empirical evidence of their effectiveness in improving predictive accuracy across various domains[25, 26, 27, 28]. The introduction of PFs extends the model's capacity to capture intricate, non-linear dependencies among features, contributing to a more comprehensive understanding of relationship dynamics. We aim to capture complex patterns such as how the frequency and intensity of interactions between users influence the strength of their relationship. This allows the model to make more accurate predictions about the nature of relationships within the network. We extract PFs from hand-crafted features representing user interactions and timestamps. We raise the original feature values to various powers, determined by the chosen polynomial degree. Let x_{ij} represent the original feature value between users i and j . The PF of degree d for x_{ij} is calculated as follows:

$$x_{ij}^d = (x_{ij})^d \quad (2)$$

These PFs are computed for each pair of interacting users, with i and j representing the indices of users involved. Given the original feature vector x , we generate PFs of degree d as follows:

$$PFs(x) = [x_1, x_2, \dots, x_n, x_1^2, x_2^2, \dots, x_n^2, \dots, x_1^d, x_2^d, \dots, x_n^d] \quad (3)$$

Next, we process PFs by the autoencoder in our model by extracting them from user interactions and timestamps. The choice of polynomial degree (d) directly impacts the model's ability to capture complex relationships. Higher degrees allow the model to capture more intricate non-linear interactions among features. However, selecting an

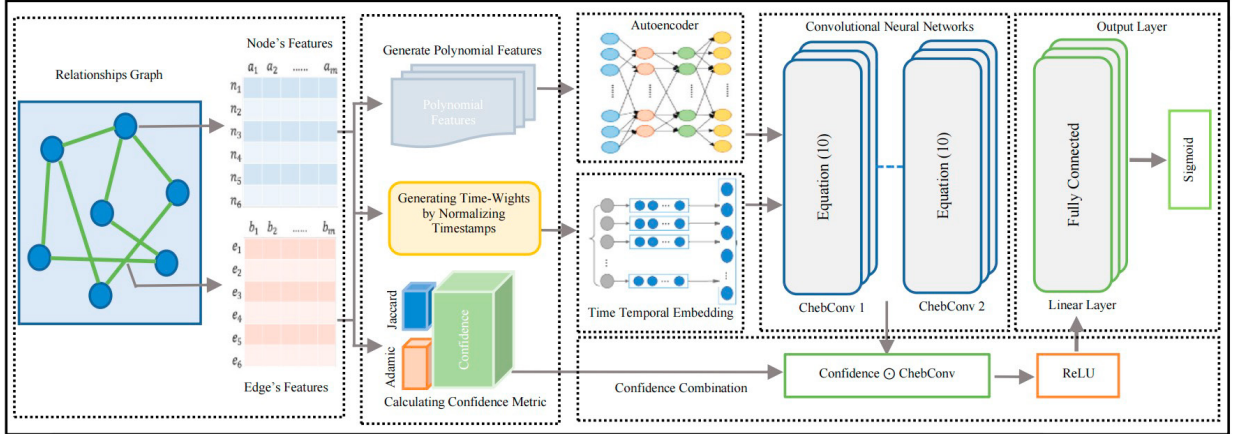


Fig. 1: The block diagram.

excessively high degree may lead to overfitting, while too low a degree may limit the model's capacity to capture nuanced relationships effectively. The best d for our model according to our experiments is discussed in section 4.

Our relation prediction GNN architecture, designed to capture temporal dynamics and complex relationships in evolving graphs, incorporates an autoencoder for feature learning, temporal embeddings for encoding time-related patterns, and a specialized TGCNConv for dynamic relationship modeling. The autoencoder plays a pivotal role in acquiring expressive representations of input features [18]. Its functionality extends to facilitating the extraction of intricate features and uncovering non-linear relationships inherent in the input data, thereby augmenting the model's capacity to discern subtle patterns. Drawing inspiration from prior works [1, 18], we employ an autoencoder architecture with the primary objective of learning expressive representations of input features. This autoencoder comprises an encoder featuring a single hidden layer equipped with a rectified linear unit (ReLU) activation function. In parallel, the decoder replicates the structural configuration of the encoder but opts for a sigmoid activation function during the reconstruction process. Specifically, the encoder $E(x)$ and decoder $D(E(x))$ are defined as follows:

$$E(x) = \text{ReLU}(W_e \cdot x + b_e) \quad (4)$$

$$D(E(x)) = \text{Sigmoid}(W_d \cdot E(x) + b_d) \quad (5)$$

Where x is the input feature vector, W_e and W_d are the weight matrices for the encoder and decoder, respectively; b_e and b_d are the bias vectors. The autoencoder operates on PFs generated from the original feature space, allowing the model to capture non-linear relationships.

Timestamp Temporal Embedding: The integration of temporal embeddings within a computational model is a significant advancement for understanding and interpreting temporal dynamics in relational data [23, 24]. We introduce the Timestamp Temporal Embedding layer to enable the transformation of discrete time attributes associated with graph edges into continuous, learnable representations. Initially, we undertake normalization of the timestamps associated with interactions to derive time weights to standardize the representation of timestamps within the range of 0 to 1, facilitating consistent scaling of temporal features. Timestamps are normalized using the formula:

$$TW = \frac{\text{timestamps}}{\max(\text{timestamps})} \quad (6)$$

This step facilitates consistent scaling of temporal features, providing a basis for meaningful interpretation. The TW derived by timestamp normalization serves as the input to the embedding layer. Let TW denote the set of edge attributes representing time weight, with $|TW|$ representing its cardinality. We initialize an embedding matrix Emb of size $|TW| \times \text{emb.size}$, where each row corresponds to the embedding vector for a unique timestamp. Emb is learned during training via backpropagation, allowing the model to adaptively capture temporal semantics. Given a time weight t from the set TW, the embedding layer performs a lookup operation to retrieve the corresponding embedding

vector from Emb. This lookup operation yields the embedding vector emb_vector_t , which encapsulates the temporal information encoded by the time weight t . we can represent the embedding lookup operation mathematically as:

$$emb_vector_t = Emb[t, :] \quad (7)$$

Here, $Emb[t, :]$ signifies the row in the embedding matrix corresponding to the time weight t , which effectively retrieves the embedding vector for that time weight. Specifically, the time embedding layer is initialized as an instance of `nn.Embedding`, is a module provided by the PyTorch library specifically designed for embedding categorical variables. This layer allows us to map discrete temporal attributes, such as timestamps associated with graph edges, to dense, lower-dimensional representations (embeddings) that can be effectively utilized in the model's computations. The `nn.Embedding` module automatically handles the creation and management of an embedding matrix, where each row corresponds to the embedding vector for a unique categorical value. The output of the embedding layer for all normalized timestamps TW forms the temporal embeddings matrix TE . Each row in this matrix encapsulates a continuous vector representation of a time weight, capturing the nuances of temporal patterns.

$$TE \in \mathbb{R}^{NumTimeEmbeddings \times EmbeddingDimension} \quad (8)$$

Algorithm 1: R-CTGCN Algorithm

Input: User interaction matrices $U_1, U_2, \dots, U_t$; Timestamp matrices $T_1, T_2, \dots, T_t$; User attribute matrices $A_1, A_2, \dots, A_t$; Polynomial degree pd ; Temporal embedding dimension ted ; Learning rate lr ; Convergence condition ξ

Output: Predicted relation scores for each user pair (u_i, u_j)

1 PRE-PROCESSING

- 2 For each user pair (u_i, u_j) , calculate Confidence Score using user attribute matrix (using Eq. 1);
- 3 Extract PFs PF from user interaction matrices using degree pd (Eq. 2 and 3);
- 4 Normalize timestamps in $T_1, T_2, \dots, T_t$ (Eq. 6);
- 5 Generate temporal embeddings TE from normalized timestamps using dimension ted ;

6 INITIALIZATION

- 7 Initialize weights and biases for Autoencoder and TGCN layers;

8 TRAINING

- 9 **while** ξ is not met **do**
- 10 Apply Autoencoder on PF to get latent feature representation LF;
- 11 Apply ChebConv layer on LF and TE (Eq. 10);
- 12 Do TGCNConv layer output using Confidence Score, ChebConv output& ReLU activation (Eq. 9);
- 13 Apply a Linear Layer to TGCNConv output and a sigmoid function to predict relations (Eq. 12);
- 14 Calculate loss using binary cross-entropy between predicted and actual relations (Eq. 13);
- 15 Update weights using Adam optimizer with learning rate lr ;
- 16 **end**

17 PREDICTION

- 18 For each user pair (u_i, u_j) , apply trained TGCN model to predict relation score;
 - 19 Map relation scores to probabilities using sigmoid function;
 - 20 Return predicted relation scores for all user pairs;
-

The temporal embedding TE is seamlessly integrated into the GNN architecture to augment node representations with temporal information. This integration equips the model with the ability to discern and leverage evolving temporal patterns, contributing to accurate relationship predictions in dynamic graphs. We propose Temporal Graph Convolutional Network (TGCNConv) that extends the Chebyshev Graph Convolutional Layer (ChebConv) [22] with the ability to consider temporal features. The layer takes into account both the input node features and the associated temporal embeddings, providing a comprehensive view of the evolving relationships over time. Also, the TGCNConv applies confidence values of users to improve the reliability of predictions. Accordingly, our proposed TGCNConv introduces innovative enhancements to the ChebConv, empowering the model with the ability to effectively incorporate confidence values and temporal dynamics.

Confidence values C , representing the reliability or trustworthiness of user interactions, are integrated into the TGCNConv layer and enable our model to dynamically weigh the influence of each interaction based on the confidence level associated with the users involved. This adaptive weighting enables the prioritization of interactions with higher confidence levels, thereby enhancing prediction accuracy and robustness, particularly in scenarios where data quality varies across interactions. Moreover, by integrating time weight embeddings TE into the TGCNConv layer, our model gains the ability to leverage temporal context when making predictions. This temporal awareness enables the model to capture long-term dependencies, account for seasonality or periodic patterns, and adapt to evolving relationships, thereby enhancing its predictive accuracy and generalization capability. The TGCNConv operation is defined as:

$$TGCNConv(X, C, A, TE) = ReLU(C \odot ChebConv(X, A, TE)) \quad (9)$$

$$ChebConv(X, A, TE) = \sum_{k=1}^{K-1} L2(X) \cdot T_k(\hat{L}) \cdot TE \quad (10)$$

In these equations, Chebyshev polynomials $T_k(\hat{L})$ are computed using the scaled Laplacian $\hat{L}$ which is derived from the adjacency matrix A . K is the number of Chebyshev polynomials. $\odot$ denotes element-wise multiplication. $L2$ normalization is applied to the input temporal features for numerical stability. $L2$ normalization of input features of two users i and j is defined as follows:

$$[L2(X)]_{ij} = \frac{X_{ij}}{\sqrt{\sum_k x_{ik}^2}} \quad (11)$$

Following the element-wise multiplication, the result undergoes the ReLU activation function. Dropout layers are then incorporated after each TGCNConv layer to augment the model's generalization capability. The features processed through the TGCNConv layer are utilized to discern the dynamics of underlying relationships. We apply a fully connected linear layer after the TGCNConv layer. This layer processes the features, and its output is fed into a sigmoid function for generating predictions. The operation of the fully connected linear layer is defined as:

$$LinearLayer(X') = WX' + b \quad (12)$$

W is the weight matrix and b is the bias vector. The output from this layer is passed through a sigmoid function $\sigma(\cdot)$ that maps the output to a range between 0 and 1, representing the probability of a relationship between the two users.

Model Training: We employ a dual strategy consisting of negative sampling [20] and the utilization of a binary cross-entropy loss [1] function to proficiently train the model. The concurrent application of the binary cross-entropy loss in conjunction with negative sampling is particularly well-suited for binary classification tasks, aligning seamlessly with our objective of predicting relationships as either present or absent. Additionally, the optimization process involves the incorporation of the Adam optimizer [19] in tandem with gradient descent for parameter updates, thereby augmenting training speed and memory efficiency. In this vein, the training regimen revolves around the optimization of the model through the deployment of the Adam optimizer in conjunction with a binary cross-entropy loss function:

$$Loss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(\hat{y}_i)) + (1 - y_i) \log(1 - \sigma(\hat{y}_i))] \quad (13)$$

N is the number of training samples, y_i and $\hat{y}_i$ denote the ground truth and predicted label, respectively, and σ symbolizes the sigmoid activation function. The training process unfolds across multiple epochs, incorporating dropout as a preventative measure against overfitting. Algorithm 1 summarizes the proposed model.

4. Experiments Design

We conducted experiments on two diverse datasets, Epinions¹ [3, 13] and Enron² [1], shown in Table 1, to evaluate the proposed approach's performance in implicit relation prediction. The datasets were partitioned into 80% training and 20% testing sets, and we employed k-fold cross-validation with k=10 for robust evaluation.

Table 1: Details of used datasets.

Datasets	Epinions	Enron
Number of nodes	158,080	98,061
Number of edges	717,667	1,148,071
No. of positive relationships	465,729	830,269
No. of relations	3	2
Explicit Relation	users' co-rating relationship	User communication relationship
Identified Implicit Relation	users' trust/distrust relationship	Manager-subordinate relationship

For our experimental setup, we tuned key hyperparameters as follows. The learning rate was set at 0.01 for 100 epochs in one configuration and 0.001 for 200 epochs in another, striking a balance between model convergence and computational efficiency. Autoencoder (AE) and Temporal Graph Convolutional Network (TGCNConv) dimensions were selected for optimal performance. The AE hidden dimensions were fixed at 64 to capture nuanced features within the data. In TGCNConv, we experimented with two settings – 64 dimensions for experiments with a learning rate of 0.01 and 100 epochs, and 128 dimensions for experiments with a learning rate of 0.001 and 200 epochs. These configurations aim to strike a balance between model expressiveness and computational efficiency. For d and e, we conduct a systematic exploration across a range of values. Figure 2 (a) and (b) showcases the impact of different d values on model performance, providing valuable insights into the optimal degree of complexity required to capture the underlying relationships within the Epinions and Enron datasets. For (d), spanning from 1 to 9, we evaluated model performance on both datasets, employing the F1 measure as the evaluative metric. Our findings revealed that a d value of 3 yielded optimal performance across varying learning rates (0.01 and 0.001) and epochs (100 and 200) for the Epinions. For the Enron, a d value of 2 exhibited superior performance across all hyperparameter configurations.

Similarly, we conducted experiments with e values ranging from 1 to 12. Figures 2 (c) and (d) provide insights into the impact of varying embedding sizes on model performance across the Epinions and Enron datasets. On the Epinions dataset, an e value of 9 consistently demonstrated the highest performance across different learning rates and epochs. Conversely, on the Enron dataset, an e value of 8 yielded optimal results. Further analysis can be gained in figure 2 (a,b) and (c,d). We experimentally establish the values of d and e as 3 and 9 for the Epinions, and 2 and 8 for the Enron dataset, respectively. While further exploration with a broader range of d and e values may yield incremental improvements, it is imperative to consider the associated increase in computational complexity. Therefore, we opted for conservative values that yielded optimal performance while maintaining computational efficiency.

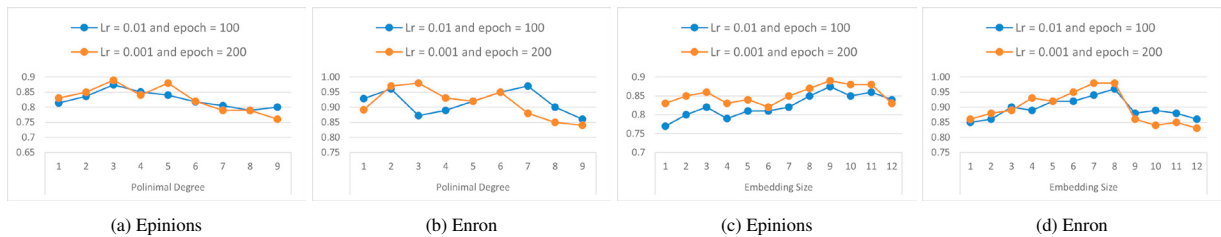


Fig. 2: Sensitive analysis of (a-b) d value on both datasets of Epinions and Enron, and (c-d) e value on both datasets of Epinions and Enron.

¹ <https://snap.stanford.edu/data/soc-Epinions1.html>

² <https://snap.stanford.edu/data/email-Enron.html>

In assessing the effectiveness of our approach, we utilized three standard evaluation metrics: Area under curve [1], Ranking score[1], F1 Score[2]. We compare the performance and effectiveness of various components using four distinct model variants, each highlighting specific aspects of our approach: **Model 1** represents the foundational layer of our approach, focusing solely on the mean combination of Timestamps temporal embedding in GCNConv. It deliberately excludes both Confidence and PFs to serve as a baseline, enabling us to measure the incremental impact of these additional components. In **Model 2** variant, PFs are omitted. This model helps in understanding the influence of the Confidence component in isolation. **Model 3** operates without the Confidence factor, thus emphasizing the role of PFs. **Model 4** is our proposed approach (R-CTGCN), which integrates the full spectrum of features and methodologies, including Confidence, PFs, and the advanced temporal embedding techniques in GCNConv. To benchmark the efficacy of our approach, we compare against two leading methods in the domain: Mirror [1] and R-GCN [10].

5. Results and Discussion

In the Enron dataset analysis, the R-CTGCN model exhibited notable performance across various configurations, as detailed in Table 2. With a learning rate of 0.01 and 100 epochs, R-CTGCN achieved the highest AUC of 0.9463 and F1 score of 0.9593, signifying its superior ability to discriminate between positive and negative instances. Furthermore, it registered a low RS of 1.06, indicative of its effectiveness in accurately ranking relationships. The model variants, excluding Confidence or PFs, or both, demonstrated diverse performances. Model 1 (lacking PFs and Confidence) yielded an AUC of 0.9076 and an RS of 1.061, whereas Model 2 (without Confidence) achieved slightly higher scores with an AUC of 0.9098 and an RS of 1.049. Model 3 (absent PFs) exhibited further enhancement with an AUC of 0.931 and an RS of 1.04, highlighting the incremental benefits conferred by each component in the model. In contrast, the Mirror and R-GCN models lagged behind with AUCs of 0.84867 and 0.801596, and RS scores of 1.065 and 1.082, respectively, underscoring the superior discrimination and ranking precision of the R-CTGCN model.

Table 2: Evaluation results on Enron dataset.

Methods	Learning rate (0.01) and epoch (100)			Learning rate (0.001) and epoch (200)		
	AUC	RS	F1	AUC	RS	F1
Model 1	0.9076	1.0610	0.9286	0.9175	1.0814	0.8986
Model 2	0.9098	1.0499	0.7978	0.9228	1.0559	0.9039
Model 3	0.9318	1.0401	0.8719	0.9385	1.0579	0.9409
R-CTGCN	0.9463	1.0600	0.9593	0.9502	1.061	0.9841
Mirror	0.8486	1.0654	0.9058	0.8531	1.2807	0.8822
R-GCN	0.8015	1.0823	0.8357	0.8371	1.0993	0.8486

In the subsequent configuration with a learning rate of 0.001 and 200 epochs, R-CTGCN maintained its dominance, achieving the highest AUC of 0.95, F1 of 0.984, and a lower RS of 1.06, underscoring its robustness across settings. Conversely, the modified models lacking Confidence, PFs, or both, exhibited inferior AUCs and F1 scores compared to the comprehensive R-CTGCN model. Against the backdrop of the Mirror and R-GCN models, with AUCs of 0.85 and 0.84, and RS of 1.28 and 1.09 respectively, the superior discriminatory and ranking capabilities of the R-CTGCN model were reaffirmed. In summary, the results conclusively establish the superiority of the R-CTGCN model in implicit relation prediction, particularly leveraging Confidence and PFs, while the comparative analysis with state-of-the-art methods further validates its efficacy in complex network scenarios.

Results from Epinions exhibit distinct trends under various configurations (Table 3). With a learning rate of 0.01 and 100 epochs, R-CTGCN attained the highest AUC of 0.901 and an F1 score of 0.874, showing better distinction between positive and negative instances. Additionally, it recorded the lowest RS of 0.76, underscoring its effectiveness in accurately ranking relationships. The model variants, excluding Confidence or PFs (PFs), or both, exhibited varied performances. Model 1 (without PFs and Confidence) yielded an AUC of 0.832 and an RS of 0.793, whereas Model 2 (without Confidence) achieved slightly higher scores with an AUC of 0.886 and an RS of 0.76. Further enhancement was observed in Model 3 (absent PFs) with an AUC of 0.887 and an F1 score of 0.86, highlighting the incremental

benefits conferred by each component in the model. In contrast, the Mirror and R-GCN models lagged with AUCs of 0.834 and 0.813, F1 scores of 0.81 and 0.80, and RS scores of 0.77 and 0.809, respectively, reinforcing the superior discrimination and ranking precision of the R-CTGCN model.

Table 3: Evaluation results on Epinions dataset.

Methods	Learning rate (0.01) and epoch (100)			Learning rate (0.001) and epoch (200)		
	AUC	RS	F1	AUC	RS	F1
Model 1	0.8327	0.7939	0.8667	0.8195	0.7766	0.8784
Model 2	0.8862	0.7691	0.7816	0.8328	0.7617	0.881
Model 3	0.8873	0.7826	0.8603	0.8897	0.7477	0.8967
R-CTGCN	0.9017	0.7647	0.8745	0.9095	0.7296	0.8873
Mirror	0.8344	0.7735	0.8173	0.8463	0.7817	0.8585
R-GCN	0.8136	0.8097	0.8052	0.8291	0.7966	0.8324

In the subsequent configuration with a learning rate of 0.001 and 200 epochs, R-CTGCN maintained its dominance, achieving the highest AUC of 0.909 and the lowest RS of 0.72, indicating its robustness across settings. Conversely, the modified models lacking Confidence, PFs, or both, exhibited inferior AUCs and F1 scores compared to the comprehensive R-CTGCN model. Against the backdrop of the Mirror and R-GCN models, with AUCs of 0.84 and 0.82, F1 scores of 0.85 and 0.83, and RS scores of 0.78 and 0.79 respectively, the superior discriminatory and ranking capabilities of the R-CTGCN model were reaffirmed. In summary, the results conclusively establish the superiority of the R-CTGCN model in implicit relation prediction, particularly leveraging Confidence and PFs, while the comparative analysis with state-of-the-art methods further validates its efficacy in complex network scenarios.

The results show the effectiveness of the R-CTGCN model for implicit relation prediction. Its performance consistently surpassed that of compared methods across various configurations. Regardless of dataset or hyperparameter settings, R-CTGCN demonstrated superior discriminative power, ranking precision, and overall predictive performance. This suggests that the integration of Confidence and PFs, along with advanced temporal embedding techniques in GCNConv, significantly enhances the model's ability to capture intricate relationships within complex networks.

Furthermore, the systematic exploration of hyperparameters, including learning rate, epoch, polynomial degree (d), and embedding size (e), provided valuable insights into their impact on model performance. Optimal values for d and e were determined through comprehensive experimentation, balancing model expressiveness and computational efficiency. By selecting conservative values that yielded optimal performance, the R-CTGCN model demonstrated robustness across different settings while maintaining computational traceability. The comparative analysis against state-of-the-art methods, namely Mirror and R-GCN, further underscored the superiority of the R-CTGCN model in implicit relation prediction. With consistently higher AUC, F1 scores, and lower RS values, R-CTGCN outperformed these methods across all configurations, reaffirming its effectiveness in complex network scenarios.

Overall, the results firmly establish the R-CTGCN model as a state-of-the-art methodology for implicit relation prediction. Leveraging Confidence and PFs, along with advanced temporal embedding techniques, R-CTGCN offers superior discriminatory power and ranking precision, promising applications in diverse domains requiring accurate relationship modeling and prediction. The comprehensive experimental design and meticulous analysis provide valuable insights into the model's effectiveness and highlight avenues for future research and optimization.

6. Conclusions, limitations and future work

In this study, we introduced the R-CTGCN, a novel approach for implicit relation prediction in dynamic networks. By integrating Confidence, PFs, and Temporal Dynamics, R-CTGCN effectively captures the complexities of evolving relationships. Our experiments on the Epinions and Enron datasets demonstrated superior performance over other state-of-the-art methods like Mirror and R-GCN. The implications of our findings are significant for the broader field of social network analysis, promising valuable applications across diverse domains such as recommendation systems, fraud detection, and social influence modeling.

While R-CTGCN shows promising results, there are three main limitations to address: First, in terms of scalability, R-CTGCN performs well on moderately sized datasets like Epinions and Enron, but its effectiveness may decrease on larger networks due to increased computational complexity and memory requirements. Second, feature engineering limits the effectiveness of our model which relies heavily on the chosen Polynomial Features and Confidence metrics that were developed based on domain knowledge and experimentation. Other potentially relevant features or combinations may enhance model performance. Third, in terms of Interpretability, as a complex neural network model, the model lacks transparency, making it challenging to trust its predictions, especially in high-stakes scenarios.

In future work, we can enhance the scalability to handle large-scale networks efficiently without sacrificing predictive performance, investigate additional feature engineering techniques, like network embeddings or attention mechanisms, and develop methods to enhance interpretability, such as feature importance analysis or activation mapping.

References

- [1] J. Liu, F. Xia, J. Ren, B. Xu, G. Pang, L. Chi, MIRROR: Mining Implicit Relationships via Structure-Enhanced Graph Convolutional Networks, *ACM Trans Knowl Discov Data*. 17 (2023). <https://doi.org/10.1145/3564531>.
- [2] J. Liu, F. Xia, L. Wang, B. Xu, X. Kong, H. Tong, I. King, Shifu2: A Network Representation Learning Based Model for Advisor-Advisee Relationship Mining, *IEEE Trans Knowl Data Eng*. 33 (2021) 1763–1777. <https://doi.org/10.1109/TKDE.2019.2946825>.
- [3] R. Barzegar Nozari, H. Koohi, Novel implicit-trust-network-based recommendation methodology, *Expert Syst Appl*. 186 (2021) 115709.
- [4] J.D. Wren, R. Bekeredian, J.A. Stewart, R. V. Shohet, H.R. Garner, Knowledge discovery by automated identification and ranking of implicit relationships, *Bioinformatics*. 20 (2004) 389–398. <https://doi.org/10.1093/BIOINFORMATICS/BTG421>.
- [5] W. Wang, J. Liu, T. Tang, S. Tuarob, F. Xia, Z. Gong, I. King, Attributed Collaboration Network Embedding for Academic Relationship Mining, *ACM Transactions on the Web (TWEB)*. 15 (2020). <https://doi.org/10.1145/3409736>.
- [6] Y. Cen, J. Zhang, G. Wang, Y. Qian, C. Meng, Z. Dai, H. Yang, J. Tang, Trust relationship prediction in Alibaba E-Commerce platform, *IEEE Trans Knowl Data Eng*. 32 (2020) 1024–1035. <https://doi.org/10.1109/TKDE.2019.2893939>.
- [7] J. Peng, L. Zhu, Y. Wang, J. Chen, Mining Relationships among Multiple Entities in Biological Networks, *IEEE/ACM Trans Comput Biol Bioinform*. 17 (2020) 769–776. <https://doi.org/10.1109/TCBB.2019.2904965>.
- [8] Z. Sun, J. Sheng, B. Wang, A. Ullah, F.R. Khawaja, Identifying Communities in Dynamic Networks Using Information Dynamics, *Entropy* 2020, Vol. 22, Page 425. 22 (2020) 425. <https://doi.org/10.3390/E22040425>.
- [9] M. Song, et al, Discovering Implicit Entity Relation with the Gene-Citation-Genes Network, *PLoS One*. 8 (2013) e84639.
- [10] Schlichtkrull, M., Kipf, T.N., Bloem, P., Van Den Berg, R., Titov, I. and Welling, M., 2018. Modeling relational data with GCN. In *The Semantic Web: 15th Int Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings* 15 (pp. 593-607). Springer Int Publishing.
- [11] Hamilton, W., Ying, Z. and Leskovec, J., 2017. Inductive representation learning on large graphs. *Advances in NIPS*, 30.
- [12] Zhang, C., Song, D., Huang, C., Swami, A. and Chawla, N.V., 2019, July. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery data mining* (pp. 793-803). <https://dl.acm.org/doi/abs/10.1145/3292500.3330961>
- [13] Nozari, R.B. and Koohi, H., 2022. An Implicit Trust-Network construction approach and a recommendation methodology for recommender systems. *Software Impacts*, 12, p.100242.
- [14] Nozari, R.B. and Koohi, H., 2020. A novel group RS based on members' influence and leader impact. *Knowledge-Based Sys*, 205, p.106296.
- [15] Nozari, R.B., Koohi, H. and Kobti, Z., 2024. IBGR: Influence-Based Group Recommendation system. *Software Impacts*, 19, p.100597.
- [16] Adamic, L.A. and Adar, E., 2003. Friends and neighbors on the web. *Social networks*, 25(3), pp.211-230.
- [17] Barzegar Nozari, R., Koohi, H. and Mahmodi, E., 2020. A novel trust computation method based on user ratings to improve the recommendation. *International Journal of Engineering*, 33(3), pp.377-386. <https://www.ije.ir/article.104607.html>
- [18] Kipf, T.N. and Welling, M., 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.
- [19] Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [20] Trouillon, T., Welbl, J., Riedel, S., Gaussier, É. and Bouchard, G., 2016, June. Complex embeddings for simple link prediction. In *International conference on machine learning* (pp. 2071-2080). PMLR. <https://proceedings.mlr.press/v48/trouillon16.html?ref=https://githubhelp.com>
- [21] Pasa, L., Navarin, N. and Sperduti, A., 2022. Polynomial-based graph CNN for graph classification. *Machine Learning*, pp.1-33.
- [22] He, M., Wei, Z. and Wen, J.R., 2022. CNNs on graphs with chebyshev approximation, revisited. *Advances in NIPS*, 35, pp.7264-7276.
- [23] Haddad, M., Bothorel, C., Lenca, P. and Bedart, D., 2020. Temporalnode2vec: Temporal node embedding in temporal networks. In *Complex Networks and Their Applications VIII: Volume 1* (pp. 891-902). Springer Int. Publishing.
- [24] Zhang, F., Zhang, Z., Ao, X., Zhuang, F., Xu, Y. and He, Q., 2022, October. Along the time: Timeline-traced embedding for temporal knowledge graph completion. In *Proceedings of the 31st ACM International Conference on Information Knowledge Management* (pp. 2529-2538).
- [25] Lingam, V., Sharma, M., Ekbote, C., Ragesh, R., Iyer, A., Sellamanickam, S. (2022, Sep). A piece-wise polynomial filtering approach for graph neural networks. In *Joint European Conf on ML and Knowledge Discovery in Databases* (pp. 412-452). Cham: Springer Int. Publishing.
- [26] Pasa, L., Navarin, N., Sperduti, A. (2022). Polynomial-based graph CNNs for graph classification. *ML*, 111(4), 1205-1237.
- [27] Guo, Y., Wei, Z. (2023, July). GNNs with learnable and optimal polynomial bases. In *International Conf on ML* (pp. 12077-12097). PMLR.
- [28] Kim, Y., Oh, H. (2021). Comparison between multiple regression analysis, polynomial regression analysis, and an artificial neural network for tensile strength prediction of BFRP and GFRP. *Materials*, 14(17), 4861. <https://doi.org/10.3390/ma14174861>